

Summary of Workshop Objectives and Techniques

The purpose of this workshop was to build and train a convolutional neural network (CNN) using PyTorch to classify images of chihuahuas and muffins. Although these two objects belong to completely different categories, they often share similar colors, textures, and shapes. These similarities make the task a challenging image classification problem. This assignment helped me understand how CNNs process image data and how deep learning models are trained and evaluated in a Jupyter Notebook environment.

The project began by loading the training and validation datasets using the torchvision library. The images were resized to a uniform size, converted into tensors, and normalized before being organized into batches using DataLoader objects. These preprocessing steps ensured that every image had consistent dimensions. They also allowed the model to train more efficiently.

The neural network consisted of several convolutional layers. These layers were followed by ReLU activation functions and max-pooling layers that gradually extracted meaningful visual features from the images. After feature extraction, the network used fully connected layers to classify each image as either a chihuahua or a muffin. Unlike the previous workshop, which used only fully connected layers, this notebook implemented a CNN architecture designed specifically for image recognition.

The model was trained using the CrossEntropyLoss function together with the Adam optimizer for ten epochs. Throughout training, both loss and accuracy were recorded for the training and validation datasets. By the end of the notebook, the model achieved a final training accuracy of approximately 93.3% and a final validation accuracy of 90.0%. The validation accuracy reached its highest value of 96.7% during the later training epochs before finishing at 90.0%. These results demonstrate that the model learned useful image features while maintaining strong overall performance.

Key Concepts Learned

This workshop significantly improved my understanding of convolutional neural networks and why they are better suited for image classification than traditional fully connected networks. I learned that convolutional layers automatically detect visual features such as edges, textures, and shapes. They also preserve the spatial relationships between pixels in an image. Pooling layers reduce the size of feature maps. This makes the model more computationally efficient while still retaining the most important information.

I also developed a better understanding of the complete deep learning workflow. Preparing image data through resizing, normalization, and batching is just as important as building the neural network itself. Using DataLoaders simplified the training process by feeding images to the model in manageable batches. It also helped by shuffling the training data before each epoch to improve learning.

Another important concept was the role of the optimizer and the loss function. `CrossEntropyLoss` measured how different the model's predictions were from the correct labels. The Adam optimizer adjusted the network's weights after every batch to gradually reduce the error. Watching both the loss and accuracy change over time helped me understand how the model improves through repeated training.

Challenges Encountered

One of the biggest challenges during this workshop was understanding how the dimensions of the data changed as images passed through each convolutional and pooling layer. At first, it was difficult to visualize how feature maps became smaller while the number of learned features increased. Reviewing the model architecture carefully helped me understand this process. Tracing the tensors through each layer also made it easier to see how CNNs extract increasingly complex visual information.

Another challenge was interpreting the training results. I noticed that validation accuracy continued to improve throughout the middle of training. It then finished slightly lower in the final epoch. This helped me realize that evaluating a model requires looking at performance across all epochs rather than focusing only on the final result. Observing both the training and validation curves gave me a better understanding of how the model learned over time.

Insights Gained

This workshop made the process of training a convolutional neural network much more concrete. Rather than simply reading about CNNs, I was able to build one, train it, and evaluate its performance using real image data. Seeing the model improve from relatively low accuracy during the first few epochs to stronger performance later in training demonstrated how neural networks gradually learn meaningful visual patterns.

I also gained a greater appreciation for how different architectural choices influence model performance. Compared with a simple neural network, the CNN was much better suited for image classification. This is because it can automatically learn spatial features directly from images. Watching the validation accuracy reach as high as 96.7% showed the effectiveness of convolutional layers for this type of task.

Real-World Applications

Convolutional neural networks are widely used in many industries that rely on image recognition. In healthcare, CNNs assist in analyzing medical images such as X-rays and MRI scans. In manufacturing, CNNs support automated quality inspection by identifying defective products. In agriculture, similar systems are used to detect plant diseases. In security systems, CNNs are used for facial recognition and object detection.

Although this workshop focused on classifying chihuahuas and muffins, the same overall workflow is used in many real-world applications. This workflow includes preprocessing images, training a convolutional neural network, and evaluating its performance. These steps form the foundation of many practical artificial intelligence systems.

Personal Reflection

This workshop strengthened both my technical skills and my confidence in using PyTorch. Building the CNN step by step helped me understand how convolutional layers, pooling layers, activation functions, and fully connected layers work together to classify images. It was satisfying to see the model improve throughout training. The model finished with approximately 93.3% training accuracy and 90.0% validation accuracy.

The feedback on this assignment also reminded me of the importance of accurately reporting experimental results. I learned that my reflection should always match the outputs produced in my notebook instead of relying on estimates or examples from other projects. In future assignments, I also want to go beyond a single training run by testing different learning rates or implementing early stopping. This would allow me to compare results from multiple experiments and support my conclusions with stronger evidence.

Overall, this workshop provided valuable experience building, training, and evaluating a convolutional neural network. It also reinforced the importance of connecting written analysis directly to experimental results.